ECDSA Private-Key Recovery from Partial Nonce Leakage

Using Neural Networks and LLL Reduction

Stadler Rareș Henea Rareș

September 21, 2026

Abstract

Elliptic Curve Digital Signature Algorithm (ECDSA) authenticates transactions and connections that use elliptic-curve keys. The security of the scheme assumes that the session nonce k is uniform, independent, and secret for every signature. When an implementation reveals information about the nonce, such as several bits, a statistical bias, or a correlated power-consumption trace, an attacker can formulate the problem using Hidden Number Problem (HNP) techniques and lattice reduction. Results reported by Breitner and Heninger [2] and by work associated with the Minerva attack [11] show that such leakage can enable private-key recovery in real systems.

The experiment studies a controlled synthetic attack chain. The current default trace generator uses a toy temporal-mixing model with Gaussian noise; earlier artifacts were generated using a Hamming Weight/Hamming Distance model and are retained as historical results. A one-dimensional convolutional neural network (NonceBitCNN), trained in PyTorch, estimates nonce-prefix bits, while the HNP stage uses these estimates in a Kannan-embedding lattice reduced with LLL/fpylll. The temporal generator is not a validated physical model of ESP32 power consumption, and no claim of key recovery from physical traces is made without measured waveform data.

The experiment is interpreted together with the arithmetic results from the companion work *Arithmetic Concurrence and Diophantine Resonance* [5]. The rational-resonance condition for exact concurrence and the quantitative near-concurrence defect $\delta_M(R)$ are used to describe the rigidity exploited by LLL within the error interval $[0, B)$ determined by the unknown nonce bits.

The verification module `theory_to_experiment_bridge8.py` tests four independent theoretical identities, including

$$\liminf_{M\to\infty} M\delta_M(\sqrt{2}) = \frac{1}{4}$$

and the irrationality-exponent transfer $\mu_{\text{geo}} = 2$. All keys, nonces, and traces used in the experiments are generated and controlled by the research team.

Keywords: ECDSA, Hidden Number Problem, LLL lattice reduction, one-dimensional convolutional neural networks, side-channel analysis, Diophantine approximation, arithmetic resonance, secp256k1.

Reproducibility note. Numerical tables generated with the earlier HW-prefix model are historical artifacts and must not be interpreted as results of the current temporal model. Current model metrics must be regenerated before being reported as current results.

Contents

1 Introduction **3**

2 Related Work **3**

3 Technical Background **4**

3.1 ECDSA over secp256k1 4

3.2 Hidden Number Problem 5

3.3 Reduction to a Lattice Problem 5

3.4 Side-Channel Leakage Model 6

4 One-Dimensional Convolutional Neural Network **6**

5 Arithmetic Results and Key-Recovery Threshold **7**

5.1 Arithmetic Results Used 7

5.2 Computational Interpretation: HNP as a Noise-Delayed Resonance Problem . . . 8

5.3 Numerical Verification of the Arithmetic Background 9

6 Experimental Methodology **9**

7 Comparative Evaluation: CNN Ensemble and Robust HNP **11**

7.1 Robust HNP Procedure 11

7.2 Scope and Limitations 12

8 Additional Experimental Validation **12**

8.1 Temporal Leakage and Model-Aware Decoding 12

8.2 HNP Sensitivity to Incorrect Prefixes 13

8.3 Confidence-Based Signature Selection 13

9 Discussion **13**

10 Ethical Considerations **14**

11 Conclusions and Future Directions **14**

1 Introduction

Elliptic Curve Digital Signature Algorithm (ECDSA) authenticates transactions and connections based on elliptic-curve keys. The scheme relies on the discrete logarithm problem on the selected curve, but its implementation must generate the nonce k correctly: uniformly, independently, and secretly for every signature. Nonce reuse, a weak random generator, or a side-channel leakage mechanism, for example through power consumption or electromagnetic emissions, can compromise this protection. The present project tests a synthetic hypothesis and does not claim a demonstrated physical side-channel attack.

When partial and noisy information about a nonce is available, such as several bits, private-key recovery can be formulated as an instance of the Hidden Number Problem (HNP), introduced formally by Boneh and Venkatesan [1]. For such instances, lattice reduction, especially the Lenstra–Lenstra–Lovász (LLL) algorithm [10], searches for a solution that remains compatible with multiple signatures.

Breitner and Heninger reported in 2019 the recovery of ECDSA private keys from signatures with biased nonces [2]. Minerva investigated timing leakage in ECDSA implementations in a different setting [11].

The project reconstructs this attack chain in a controlled environment. The computational part has three components:

(i) construction of an ECDSA simulator over secp256k1 without external cryptographic dependencies;

(ii) a toy temporal leakage model with Gaussian noise, together with a one-dimensional convolutional neural network trained to estimate nonce-prefix bits from synthetic traces;

(iii) an HNP solver based on Kannan embedding in a full lattice, reduced with LLL, which validates private-key candidates.

The paper then connects this computational chain to the arithmetic results in the companion work *Arithmetic Concurrence and Diophantine Resonance* [5]. That work shows that k families of lines parameterized by integer labels are exactly concurrent when their slopes satisfy a rational relation, and describes the best approximation in the irrational case through the constant $\mathcal{L}(R)$. Here, the same notion of rigidity provides a framework for the threshold at which LLL reduction can separate the correct solution from other solutions in the error interval $[0, B)$ determined by the unknown nonce bits.

2 Related Work

Hidden Number Problem and ECDSA/(EC)DSA attacks. Boneh and Venkatesan [1] introduced HNP for studying partial information about secret keys in cryptographic schemes. Howgrave-Graham and Smart [6] adapted the framework to (EC)DSA and showed how partial nonce leakage can enable private-key recovery given a sufficient number of signatures. Breitner and Heninger [2] analyzed attacks on public ECDSA signatures with biased nonces, including cases where the bias appears as a statistical correlation. Minerva and TPM-Fail studied leakage associated with timing variation and implementations of cryptographic operations [11, 12].

Lattice reduction. The LLL algorithm [10] reduces integer lattice bases and produces short vectors for moderate dimensions. BKZ can obtain stronger reductions, although its computational cost depends on the block parameter. The fpylll library provides implementations of these methods, and the project uses it in Section 3.3.

Power analysis and deep-learning side channels. Modeling power consumption with Hamming Weight and Hamming Distance is standard in differential power analysis (DPA) [8]. Convolutional networks can classify power traces without manual feature extraction, and the side-channel literature contains related approaches [3]. The architecture in Section 4 retains this idea in a compact form adapted to the present experiments.

Diophantine approximation and arithmetic resonance. The mathematical background in Section 5 comes from the companion work [5] and from classical Diophantine approximation literature [4, 7, 13]. The project also compares these results with recent work on affine copies of discrete patterns in sets of integers [9].

3 Technical Background

3.1 ECDSA over secp256k1

The secp256k1 curve [14] is defined over the finite field $\mathbb{F}_p$ by the Weierstrass equation

$$y^2 \equiv x^3 + 7 \pmod{p}, \qquad p = 2^{256} - 2^{32} - 977,$$

with generator point G of prime order n, which is the subgroup used for signing. The implementation `dataset_generator3.py` reproduces point arithmetic on the curve, including addition, doubling, and scalar multiplication by double-and-add, without external cryptographic dependencies.

The signing algorithm is:

(1) choose a private key $d \in [1, n-1]$ and compute the public key $Q = dG$;

(2) for a message with hash $z = H(m)$, choose a nonce $k \in [1, n-1]$ uniformly and compute $R = kG = (x_R, y_R)$;

(3) compute $r = x_R \bmod n$ and

$$s = k^{-1}(z + rd) \pmod{n};$$

(4) the signature is the pair (r, s).

For HNP analysis, the signing equation can be written as

$$d \equiv r^{-1}(sk - z) \pmod{n} \quad \Longleftrightarrow \quad k \equiv s^{-1}(z + rd) \pmod{n}. \tag{1}$$

When part of k is known or estimated with noise for several signatures (r_i, s_i, z_i), Equation (1) leads to a system of modular linear relations with bounded errors in the unknown d, which is precisely the structure used by HNP.

3.2 Hidden Number Problem

Definition 3.1 (Hidden Number Problem)**.** *Let q be a large prime, or the order of a large subgroup, let $\alpha \in \mathbb{Z}_q$ be an unknown secret, and let m pairs $(t_i, u_i) \in \mathbb{Z}_q^2$ be chosen uniformly so that*

$$u_i \equiv \alpha t_i + \varepsilon_i \pmod{q}, \qquad |\varepsilon_i| < B,$$

for unknown but bounded errors ε_i. HNP asks for the recovery of α from $\{(t_i, u_i)\}_{i=1}^m$ when only the bound $B \ll q$ is known.

In the implementation `hnp_attack1.py`, for each public signature the quantities

$$t_i = s_i^{-1} r_i \pmod{q}, \qquad u_i = s_i^{-1} z_i \pmod{q}, \qquad a_i = \bar{k}_i \, 2^{n_{\text{bits}} - \ell}$$

are computed, where $\bar{k}_i$ is the estimated prefix of ℓ nonce bits produced by the neural network or, for validation, taken from ground truth. Here $q = n$ is the order of the secp256k1 subgroup, and the private key d plays the role of the secret α.

From (1),

$$t_i d + u_i - a_i \equiv \delta_i \pmod{q}, \qquad 0 \le \delta_i < B,$$

where

$$B = 2^{n_{\text{bits}} - \ell}$$

represents the uncertainty introduced by the unknown lower nonce bits.

3.3 Reduction to a Lattice Problem

The HNP instance is transformed into a lattice problem using Kannan embedding. The lattice has dimension $m + 2$, where m is the number of signatures. The implementation `Lattice_key6.py`, through the class `HNPLatticeSolver`, constructs a basis of the form

$$L = \begin{pmatrix} q & & & & \\ & \ddots & & & \\ & & q & & \\ qt_1 & \cdots & qt_m & B & \\ qc_1 & \cdots & qc_m & & M \end{pmatrix}, \qquad c_i = (a_i - u_i) \bmod q, \tag{2}$$

where $B = 2^{n_{\text{bits}} - \ell}$ bounds the error in each coordinate and $M = qB$ is the embedding factor for the final row.

For the true private key d, the vector

$$v^\star = (q\delta_1, \ldots, q\delta_m, dB, -M), \qquad \delta_i = (t_i d + u_i - a_i) \bmod q,$$

is an integer combination of the basis rows in (2). Its norm satisfies approximately

$$\|v^\star\| \approx \sqrt{m}\, qB,$$

and becomes small relative to typical lattice vectors when B is sufficiently small compared with the number of signatures and the number of known bits. LLL, applied through `fpylll.LLL.reduction` or, alternatively, `sympy.Matrix.lll`, produces a reduced basis. The function

`recover_from_reduced_basis` searches for a private-key candidate, and `validate_candidate` checks it independently against all (t_i, u_i, a_i) pairs.

Algorithm 1. Private-key recovery using HNP + LLL.

Input: public signatures $\{(r_i, s_i, z_i, \ell)\}_{i=1}^{m}$ and estimated prefixes $\{\bar{k}_i\}$.
1. Compute t_i, u_i, a_i according to Section 3.2.
2. Construct the lattice basis L from (2).
3. Compute $L' \leftarrow \mathrm{LLLReduce}(L, \delta = 0.75)$.
4. For every reduced basis vector v, test whether $|v_{m+1}| = M$ and $v_m \bmod B = 0$.
5. Form $d_{\mathrm{cand}} = \pm(v_m/B) \bmod q$.
6. Validate d_{cand} against every (t_i, u_i, a_i) pair.
7. Return the candidate if validation succeeds; otherwise report failure and increase the number of signatures or reduce the noise.

3.4 Side-Channel Leakage Model

To model a synthetic power trace associated with nonce computation, `ecdsa_leakage_model4.py` uses Hamming Weight (HW) and Hamming Distance (HD) models [8]:

$$HW(v) = \mathrm{popcount}(v), \qquad HD(v_1, v_2) = \mathrm{popcount}(v_1 \oplus v_2).$$

The models are applied progressively, bit by bit, to the partial nonce prefix k. Gaussian noise

$$\mathcal{N}(0, \sigma^2)$$

is added to the clean trace. Signal quality is reported through

$$SNR_{\mathrm{dB}} = 10 \log_{10} \frac{\mathrm{Var(clean\ trace)}}{\sigma^2}.$$

The script also includes a linear reference decoder based on template matching for the two bit hypotheses. This provides a baseline before neural-network training.

4 One-Dimensional Convolutional Neural Network

For decoding nonce bits from noisy traces, the project uses a compact one-dimensional convolutional network implemented in PyTorch in `informatica/cnn/model.py`:

```
class NonceBitCNN(nn.Module):
    def __init__(self, in_channels=1, out_channels=16, kernel_size=9):
        super().__init__()
        padding = kernel_size // 2
        self.conv1 = nn.Conv1d(in_channels, out_channels,
                               kernel_size, padding=padding)
        self.relu = nn.ReLU()
        self.conv2 = nn.Conv1d(out_channels, 2, 1)
```

```
def forward(self, x):
    x = self.relu(self.conv1(x))
    return self.conv2(x)
```

ECDSA Simulator, secp256k1 ⟶ HW/HD Leakage Model ⟶ Gaussian Noise ⟶ CNN 1D, NonceBitCNN ⟶ HNP Instance (t_i, u_i, a_i) ⟶ LLL/fpylll Reduction ⟶ Private-Key Candidate and Validation

Figure 1: Computational pipeline for nonce-bit recovery and HNP-based private-key validation.

The constructor listing above shows the model's default arguments. The reference run overrides those defaults through `informatica/cnn/config.yaml`, setting `in_channels=2`, `out_channels=8`, `kernel_size=3`, and `num_classes=2`. The input contains the raw trace and its first difference. The configured sequence length is 12 positions, matching the 12 nonce bits evaluated by the reference pipeline. The final 1×1 convolution produces two logits for each position, one for each bit class.

Training minimizes the per-position cross-entropy between the predictions and the binary ground-truth labels `bit_labels`, using `profile_dataset.json` split into training and validation subsets. `cnn_nonce_analysis5.py` consumes the network output, extracts the first ℓ predicted bits, and converts them into the prefix $\bar{k}_i$ used by the HNP attack in Sections 3.2 and 3.3.

5 Arithmetic Results and Key-Recovery Threshold

This section connects the arithmetic results from [5] with the behavior of LLL reduction in Section 3.3. The analysis focuses on the dependence on the number of leaked bits ℓ and the number of signatures m.

5.1 Arithmetic Results Used

The companion work considers k families of parallel lines, each labeled by an integer $m_i \in \mathbb{Z}$:

$$x - \kappa_i y = m_i, \qquad i = 1, \ldots, k,$$

and gives the following exact concurrence criterion.

Theorem 5.1 (Exact concurrence criterion)**.** *Let $\kappa_1, \kappa_2, \kappa_3$ be distinct and let $\mathbf{m} \in \mathbb{R}^3$. The three lines $x - \kappa_i y = m_i$ are concurrent if and only if*

$$(m_1 - m_3)(\kappa_2 - \kappa_3) = (m_2 - m_3)(\kappa_1 - \kappa_3).$$

Corollary 5.2 (Arithmetic resonance)**.** *Let*

$$R = \frac{\kappa_1 - \kappa_3}{\kappa_2 - \kappa_3}.$$

There is a nontrivial concurrence with integer labels if and only if $R \in \mathbb{Q}$. If $R = p/q$ is in lowest terms, every integer pair of concurrent differences has the form

$$t(p, q), \qquad t \in \mathbb{Z}.$$

For $R \notin \mathbb{Q}$, exact concurrence with integer labels is impossible. The companion work measures the best approximation in a finite box $[-M, M]^3 \cap \mathbb{Z}^3$ through

$$\delta_M(R) = \min_{\substack{(p,q)\in\mathbb{Z}^2\setminus\{(0,0)\} \\ w(p,q)\le 2M}} |p - Rq|,$$

where

$$w(p, q) = \max\{0, p, q\} - \min\{0, p, q\}.$$

It obtains the asymptotic identity

$$\liminf_{M\to\infty} M\delta_M(R) = \frac{D(R)}{2}\mathcal{L}(R),$$

where

$$D(R) = \max\{0, 1, R\} - \min\{0, 1, R\}, \qquad \mathcal{L}(R) = \liminf_{q\to\infty} q\|qR\|.$$

5.2 Computational Interpretation: HNP as a Noise-Delayed Resonance Problem

The framework above has a structural analogy with the HNP instance in Section 3.2. Consider a single HNP relation

$$td + u - a \equiv \delta \pmod{q}, \qquad 0 \le \delta < B.$$

Following the exact-concurrence criterion, the relation can be viewed as a compatibility condition between integer-valued constraints in the (d, δ) representation modulo q. The admissible error is restricted to $\delta \in [0, B)$.

With m signatures, the same unknown d must satisfy m approximate constraints simultaneously, while

$$B = 2^{n_{\text{bits}}-\ell}$$

sets the error bound. Three consequences follow directly:

- the number of leaked bits ℓ directly controls B. As ℓ increases, B decreases and the admissible error interval becomes narrower;
- the number of signatures m increases the number of independent constraints that the candidate d must satisfy simultaneously. In the analogy with simultaneous approximation in the companion work, each additional signature restricts the compatible solution space;
- the short vector $v^\star$ in (2) plays a role analogous to the minimizing pair (p, q) in the finite-box approximation problem. LLL can identify it when its norm remains sufficiently small relative to typical vectors of the lattice basis. The constant $\mathcal{L}(R)$ provides a reference for the separation between the compatible solution and weaker approximations.

Remark 5.3. *The analogy does not constitute a formal reduction between the two problems. HNP remains, in general, a problem connected to the Shortest Vector Problem rather than to one-dimensional Diophantine approximation. The analogy is useful as an interpretive framework because both problems require an unknown bounded integer quantity to satisfy several approximate linear constraints simultaneously. In this view, the experimental threshold depends on the number*

of samples and the noise level, and this dependence can be compared with the rigidity described by $\mathcal{L}(R)$ and $\mu(R)$ in [5].

5.3 Numerical Verification of the Arithmetic Background

The numerical verification of the arithmetic theorem uses four independent modules in `theory_to_experiment_bridge8.py`. They compare theoretical values with results obtained through direct scans over finite boxes $[-M, M]$.

Table 1: Results of the four theory-to-experiment verification modules.

Module	Quantity verified	Reference value
1. Asymptotic constant	$\liminf_M M\delta_M(\sqrt{2})$ and $\liminf_M M\delta_M(\varphi)$	$1/4$, respectively $\varphi/(2\sqrt{5})$
2. Exponent transfer	Slope of $\log(1/\delta_M)$ versus $\log M$ for $R = \sqrt{2}$	$\mu_{\text{geo}} = \mu = 2$
3. Steinhaus–Sós rigidity	Number of distinct gap lengths in $\{n\sqrt{2} \bmod 1\}$	At most 3 for $N \in \{10^3, 5 \times 10^3, 2 \times 10^4\}$
4. Closure dictionary, $k = 3$	Integer solutions of $2m_1 + m_2 - 3m_3 = 0$ versus geometric intersections on $[-15, 15]^3$	321 solutions, symmetric difference 0

All four verification modules report `PASS`, and the numerical results agree with the identities from [5] used in Section 5. The corresponding convergence plots can be generated from the project data using the verification script.

6 Experimental Methodology

The experimental protocol uses five stages defined in the project plan [15] and implemented in the `informatica/` directory:

E1. Data generation. `dataset_generator3.py` generates random ECDSA keys and signatures on synthetic messages. The public dataset contains the message, r, s, and the number of leaked bits ℓ. The separate synthetic ground-truth dataset contains the private key d and complete nonce k values for reference scoring. This is a file-level separation, not an access-control barrier; the attack path must use public records and predicted prefixes only, and may consult ground truth only after the solver finishes.

E2. Leakage modeling. The current dataset pipeline produces synthetic HW/HD traces with additive Gaussian noise. The main pipeline uses 160 signatures, 12 leaked MSB nonce bits, and $\sigma = 0.15$, with a fixed split of 96 training, 24 validation, and 40 test samples. A separate parameter sweep evaluates

$$\ell \in \{8, 10, 12, 14\}, \qquad \sigma \in \{0, 0.05, 0.10, 0.15, 0.20\}, \qquad m = 40,$$

using 100 seeds, $\{1000, \ldots, 1099\}$, and 100 training epochs. The results are aggregated over the 100 runs for each configuration.

E3. CNN training. `cnn/train.py` and `cnn_nonce_analysis5.py` train the NonceBitCNN on profiled traces and then run it on the test set to produce estimated prefixes $\bar{k}_i$.

E4. HNP + LLL attack. `hnp_attack1.py` and `Lattice_key6.py` construct the HNP instance from estimated prefixes. For implementation validation, the same attack is also evaluated with ground-truth prefixes using the oracle source. The lattice is reduced with LLL through `fpylll`, and a private-key candidate is accepted only after independent validation against the HNP relations.

E5. Validation and comparison. The results are compared qualitatively with thresholds reported in the reference literature [2, 11], particularly with respect to the number of leaked bits and signatures associated with complete key recovery.

The automated test suite in `tests/` runs through the continuous integration workflows. It covers ECDSA arithmetic, HNP behavior and regressions, the mathematical verification modules, model behavior, simulation, and pipeline smoke tests. The reproducibility workflow also requires the `fpylll` backend and verifies successful HNP recovery for the fixed seed

$$\texttt{RANDOM_SEED} = 20260919.$$

Table 2: Aggregated recovery results for $m = 40$. Each row aggregates 100 seeds for one (ℓ, σ) configuration. Accuracy values are means over those runs.

ℓ	σ	CNN bit acc.	CNN prefix acc.	Oracle HNP	CNN→HNP
8	0.00	99.80%	98.45%	0/100	0/100
8	0.05	99.71%	97.70%	0/100	0/100
8	0.10	99.59%	96.75%	0/100	0/100
8	0.15	98.98%	92.45%	0/100	0/100
8	0.20	97.03%	79.60%	0/100	0/100
10	0.00	99.78%	97.88%	100/100	61/100
10	0.05	99.74%	97.50%	100/100	51/100
10	0.10	99.66%	96.70%	100/100	47/100
10	0.15	99.02%	90.80%	100/100	6/100
10	0.20	97.13%	76.40%	100/100	0/100
12	0.00	99.85%	98.28%	100/100	69/100
12	0.05	99.75%	97.00%	100/100	44/100
12	0.10	99.70%	96.50%	100/100	33/100
12	0.15	99.08%	90.00%	100/100	4/100
12	0.20	97.06%	71.60%	100/100	0/100
14	0.00	99.87%	98.35%	100/100	70/100
14	0.05	99.80%	97.23%	100/100	49/100
14	0.10	99.69%	95.70%	100/100	40/100
14	0.15	99.14%	88.98%	100/100	3/100
14	0.20	96.98%	67.82%	100/100	0/100

The table uses the recorded values in `results/tables/cnn-hnp-parameter-sweep-summary.csv`. The fixed reference pipeline run and the aggregated parameter sweep answer different questions. A key recovered in the fixed reference instance does not imply successful recovery for every seed or noise configuration. Oracle HNP uses the known prefixes and succeeds in all 100 runs for $\ell \geq 10$. CNN-to-HNP recovery depends on the predicted prefix and decreases as the noise increases.

Protocol note on the archived sweep. The archived parameter-sweep CSVs were produced by an earlier version of the orchestration code that checked whether the known synthetic private

key satisfied the HNP bounds before calling LLL. The current benchmark code has been corrected to execute the solver first and use ground truth only for post-hoc scoring. The historical recovery counts remain failure counts when the true key violates the same bounds enforced by candidate validation; however, runtime fields for cases skipped by the earlier code are conditional and must not be interpreted as end-to-end attack runtimes. Re-run the full sweep with the current script before using runtime aggregates as a performance claim.

7 Comparative Evaluation: CNN Ensemble and Robust HNP

A separate comparison evaluates whether combining CNN predictions and allowing a robust HNP search improves recovery. This experiment uses 100 seeds (1200–1299), $\ell = 12$, $\sigma = 0.10$, $m = 40$, 160 synthetic samples per seed, and 100 training epochs. The ensemble averages bit-class probabilities from five CNN models trained with different initializations on the same data split. The first model is also evaluated as the single-CNN baseline.

Table 3: End-to-end synthetic key-recovery comparison over 100 seeds.

Method	**Keys recovered**	**Rate**
Single CNN + standard HNP	43/100	43%
CNN ensemble + standard HNP	54/100	54%
CNN ensemble + robust HNP	90/100	90%
Oracle prefixes + HNP	100/100	100%

The mean test bit accuracy increased from 99.758% for the single CNN to 99.871% for the ensemble. Mean exact-prefix accuracy increased from 97.125% to 98.450%. In this configuration, ensembling improved key recovery by 11 percentage points relative to the single-CNN baseline; the robust HNP procedure added a further 36 percentage points relative to ensemble plus standard HNP.

7.1 Robust HNP Procedure

The robust method first runs HNP using all 40 signatures. If the candidate does not verify against the public ECDSA key, it tries subsets obtained by removing one signature at a time. The synthetic private key is not used to select subsets or accept a candidate. In the 100-seed dataset, the robust method recovered the key in all 54 experiments with 40/40 exact prefixes and in all 36 experiments with 39/40 exact prefixes. The 10 remaining experiments, with at least two erroneous prefixes, did not recover the key.

The method used a mean of 12.11 LLL attempts per experiment, with a maximum of 41; mean robust-HNP runtime was approximately 0.656 seconds per experiment, excluding CNN training. The verifier checks the candidate against the public key using the project's curve arithmetic implementation, so it should not be described as verification by an independent cryptographic library.

7.2 Scope and Limitations

These figures describe only the specified synthetic leakage model, noise level, sample count, and seed range. They are not a 90% recovery claim for physical power traces or arbitrary ECDSA implementations. The robust procedure tests the full set and then subsets that omit at most one signature; it is not designed to correct arbitrary numbers of wrong prefixes. The raw and aggregated results are provided in `results/validation/ensemble-robust-100-seeds/rosef-comparison-100-seeds.csv` and `results/validation/ensemble-robust-100-seeds/rosef-comparison-100-seeds-summary.csv`.

8 Additional Experimental Validation

To complement the cumulative Hamming-weight leakage experiments, we conducted additional evaluations of temporal leakage decoding, nonce-prefix error sensitivity, and confidence-based signature selection. These experiments examine the limitations of the CNN-to-HNP pipeline under a more challenging synthetic observation model. Results from the different leakage models are reported separately and are not treated as directly interchangeable.

8.1 Temporal Leakage and Model-Aware Decoding

The alternative temporal leakage model is defined by

$$L_i = 0.2b_i + 0.6b_{i-1} + 0.2b_{i-2} + \varepsilon_i, \qquad \varepsilon_i \sim \mathcal{N}(0, 0.15^2), \tag{1}$$

where b_i denotes a nonce bit and the preceding bits are set to zero at the beginning of the sequence.

The benchmark uses 160 synthetic ECDSA samples, divided into 96 training samples, 24 validation samples, and 40 test samples. All three subsets are disjoint by sample identifier. The same dataset and split are reused across 100 seeds (1400–1499), while the noise realizations and CNN training initializations vary. Consequently, these runs are not 100 independent private-key experiments.

We evaluated a threshold decoder, an exact-model Viterbi decoder, and three compact CNN variants using raw leakage, first differences, or both input channels. The mean test results over the 100 seeds are summarized in Table 4.

Table 4: Temporal leakage decoding at $\sigma = 0.15$ over 100 seeds, with 40 test signatures per seed.

Method	Bit accuracy (%)	Exact-prefix accuracy (%)
Threshold	54.41	0.00
Viterbi (known model)	93.95	56.55
CNN, raw channel	85.66	12.85
CNN, difference channel	69.49	2.02
CNN, both channels	88.86	24.95

Viterbi achieved the highest mean accuracy. Unlike the CNN models, this decoder explicitly uses the coefficients of the synthetic leakage model. Its performance therefore represents a model-informed baseline rather than a comparison under identical prior knowledge.

8.2 HNP Sensitivity to Incorrect Prefixes

We evaluated the lattice recovery procedure using controlled corruptions of oracle nonce prefixes. Each trial used 40 signatures and 12 leaked most-significant nonce bits. The number of corrupted prefixes was varied, with 10 repetitions per configuration.

With no corrupted prefixes, both the standard and robust HNP procedures recovered the private key in 10/10 trials. With one corrupted prefix, the standard procedure recovered the key in 0/10 trials, while the robust procedure recovered it in 10/10 trials. With two or more corrupted prefixes, neither procedure recovered the key in the tested trials.

The robust procedure tests the complete signature set and subsets obtained by removing one signature at a time. It should not be interpreted as a general-purpose decoder for an arbitrary number of incorrect nonce prefixes. These observations characterize the tested solver and configuration, not a universal information-theoretic recovery threshold.

8.3 Confidence-Based Signature Selection

We also ranked the 40 test signatures by the sum of logarithmic maximum class probabilities assigned by the CNN to the predicted nonce-prefix bits. This ranking does not require oracle labels at selection time.

For subsets of 20 signatures, confidence-based selection achieved a mean exact-prefix accuracy of 35.30%, compared with 23.80% for random selection. The corresponding accuracies for subsets of 25, 30, and 35 signatures were 32.88%, 30.53%, and 27.69%, respectively, compared with 23.64%, 24.13%, and 24.49% for random selection. At 40 signatures, both methods necessarily select the entire test set and obtain the same mean accuracy of 24.95%.

Although confidence-based selection improved prefix accuracy for smaller subsets, the remaining incorrect prefixes were too numerous for the evaluated standard and single-exclusion HNP recovery procedures. Therefore, improved prefix prediction should not be equated with successful private-key recovery.

9 Discussion

In the parameter sweep, CNN bit and prefix accuracy decrease as σ increases. For $\ell = 12$ and $m = 40$, mean prefix accuracy decreases from 98.28% at $\sigma = 0$ to 71.60% at $\sigma = 0.20$. Across those settings, oracle HNP recovers 100/100 keys, while CNN-to-HNP recovery falls from 69/100 to 0/100. This comparison separates the lattice stage with correct prefixes from errors introduced by CNN predictions.

For $m = 40$ and $\ell = 8$, the sweep records no recovery with either oracle or CNN-derived prefixes. For $\ell \in \{10, 12, 14\}$, oracle recovery is 100/100, while CNN-to-HNP recovery varies with noise. These measurements apply to the tested synthetic configurations and do not establish a general recovery threshold.

This dependence is consistent with the role of additional constraints discussed in [5]. Remark 5.4 of the companion work identifies a limitation of the current framework: for simultaneous approximation with $k > 3$ families, and for $m > 2$ signatures in the HNP formulation, there is not yet an exponent-transfer result with the same precision as the scalar theorem. Such a result

would provide a mathematical estimate of the minimum number of signatures as a function of ℓ and the noise level σ.

10 Ethical Considerations

The research team generates and controls all private keys, nonces, and traces used in the project; all data in this study are synthetic, and no third-party cryptographic material or production system is targeted. In the distributed archive, the explicit private-key field is redacted, but the oracle reference file retains complete synthetic nonce values. Because complete nonces and ECDSA signatures can reveal the corresponding synthetic private key, this file is evaluation-only ground truth, not a blinded attack input. The attack benchmark must not read it before solver execution; recovered candidates are checked against public-key data and ground truth is used only for post-hoc scoring.

11 Conclusions and Future Directions

This paper presents an experimental chain for studying ECDSA private-key recovery in the presence of partial nonce leakage. The chain includes an ECDSA simulator, a Gaussian-noise leakage model, a one-dimensional CNN, and an HNP solver based on Kannan embedding and LLL reduction.

The arithmetic results from [5] provide a framework for describing the recovery threshold. The exact-concurrence theorem characterizes rational resonance, while the quantitative-resonance theorem quantifies the near-concurrence defect. In the experiment, these results are used to describe the rigidity required for the correct solution to separate from other vectors in the HNP lattice.

Three future directions follow from the present framework:

(a) formulate and prove a result for the simultaneous case $m > 2$, following the higher-dimensional generalization in [5];

(b) validate the computational chain on physical traces measured from an ESP32 microcontroller, following the experimental plan in `informatica/esp32/`;

(c) extend the parameter sweep with additional seeds, leakage models, and noise regimes in order to characterize the observed recovery behavior beyond the current synthetic configurations.

References

[1] D. Boneh and R. Venkatesan. *Hardness of computing the most significant bits of secret keys in Diffie–Hellman and related schemes.* Advances in Cryptology, CRYPTO 1996, LNCS 1109, pp. 129–142, Springer, 1996.

[2] J. Breitner and N. Heninger. *Biased nonce sense: lattice attacks against weak ECDSA signatures in cryptocurrencies*. Financial Cryptography and Data Security, FC 2019, LNCS 11598, Springer, 2019.

[3] E. Cagli, C. Dumas, and E. Prouff. *Convolutional neural networks with data augmentation against jitter-based countermeasures*. Cryptographic Hardware and Embedded Systems, CHES 2017, Springer, 2017.

[4] J. W. S. Cassels. *An Introduction to Diophantine Approximation*. Cambridge University Press, 1957.

[5] Henea Rareș. *Arithmetic Concurrence and Diophantine Resonance: From Exact Integer Lattices to Quantitative Near-Concurrence*. Companion paper, RoSEF 2026–2027 project, September 2026.

[6] N. Howgrave-Graham and N. P. Smart. *Lattice attacks on digital signature schemes*. Designs, Codes and Cryptography, 23(3):283–290, 2001.

[7] A. Khintchine. *Einige Sätze über Kettenbrüche, mit Anwendungen auf die Theorie der Diophantischen Approximationen*. Mathematische Annalen, 92:115–125, 1924.

[8] P. Kocher, J. Jaffe, and B. Jun. *Differential power analysis*. Advances in Cryptology, CRYPTO 1999, LNCS 1666, Springer, 1999.

[9] S. Korsky. *Affine Copies of Three-Point Patterns in Sets of Integers*. arXiv:2609.02308, 2026.

[10] A. K. Lenstra, H. W. Lenstra Jr., and L. Lovász. *Factoring polynomials with rational coefficients*. Mathematische Annalen, 261(4):515–534, 1982.

[11] J. Jančár, V. Sedláček, P. Švenda, and M. Sýs. *Minerva: The Curse of ECDSA Nonces: Systematic Analysis of Lattice Attacks on Noisy Leakage of Bit-Length of ECDSA Nonces*. IACR Transactions on Cryptographic Hardware and Embedded Systems, 2020(4):281–308, 2020. DOI: `https://doi.org/10.13154/tches.v2020.i4.281-308`.

[12] D. Moghimi, B. Sunar, T. Eisenbarth, and N. Heninger. *TPM-Fail: TPM meets Timing and Lattice Attacks*. In 29th USENIX Security Symposium (USENIX Security 20), pages 2057–2073, 2020.

[13] W. M. Schmidt. *Diophantine Approximation*. Lecture Notes in Mathematics 785, Springer, 1980.

[14] Standards for Efficient Cryptography Group. *SEC 2: Recommended Elliptic Curve Domain Parameters*, Version 2.0, 2010.

[15] Stadler Rareș and Henea Rareș. *Research Plan for Practical Cryptographic Key Recovery*. Project planning document, RoSEF 2026–2027.